

CS 411 Lab 1

Rouge-Like Algebra

Bryan Baker

6 October 2025

Version 1

Table of Contents

Listing of Tables.....	2
1 Introduction.....	3
1.1 Societal Problem	3
1.2 Solution Characteristics	3
2 Product Overview	4
2.1 Key Product Features and Capabilities	4
2.1.1 Gameplay	4
2.1.2 Engagement.....	4
2.1.3 Objective	4
3 Major Components.....	5
3.1 Integrated Development Environment (IDE).....	5
3.2 Version Control	5
3.3 Target Operating System.....	5
3.4 Game Engine.....	5
3.5 Programming Language.....	5
3.6 Systems	5
4 Identification of Case Study.....	6
4.1 Primary Audience.....	6
4.2 Secondary Audience.....	6
5 Glossary	7
6 References.....	8

Listing of Figures

Figure 1: Current Process Flow	3
--------------------------------------	---

Listing of Tables

No table of figures entries found.

1 Introduction

There are many societal issues that people experience, and one of them is the ways that students struggle in learning algebraic concepts. Given this, we present our solution to this problem that people face.

1.1 Societal Problem

Students in the modern-day struggle to understand what is to be expected of them when it comes to Algebraic understanding. It has gotten worse in recent years due to the COVID-19 pandemic's effects when it comes to how students are handling school as well as some traditional teaching methods coming short of achieving the goal of effective understanding. This leads to a wide host of issues that have both immediate and long-term side effects if not addressed effectively including frustration for both the student and instructor, disengagement because of misunderstanding, and lack of confidence when it comes to academic endeavors, especially with math. Figure 1 demonstrates a typical flow with student understanding, and potential outcomes.

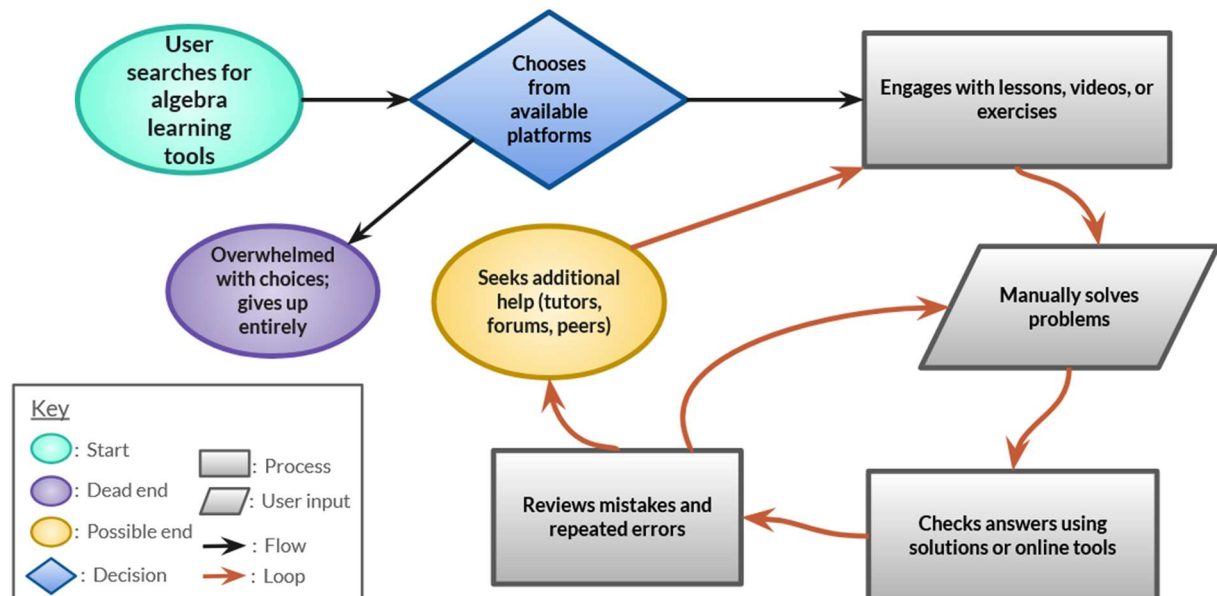


Figure 1: Current Process Flow

1.2 Solution Characteristics

To address such an issue surrounding algebraic understanding, a solution must address these underlying issues. The solution should be interactive, engaging, and include adaptive learning tools to help curb the current problem and should be distinct from the traditional methods of teaching, as they are currently unsatisfactory when dealing with the societal problem that has been made present.

“Hierarchical linear modeling analyses of the final analytic sample (N = 1,850) showed significantly higher posttest scores for students who used From Here to There and DragonBox 12+ compared to the Active Control condition.” - The Impacts of Three Educational Technologies on Algebraic Understanding in the Context of COVID-19

The quote describes a study in which different methods of teaching algebraic concepts to seventh graders in the context of COVID-19. The active control condition referred to in the study refers to the traditional means of teaching algebraic concepts, while both “From Here to There” and “DragonBox 12+” are both game-based applications that are based on teaching algebraic concepts via a gamified teaching experience. The study shows that the group of students who were selected to use the gamified experiences showed higher test scores in comparison to those students who were using traditional methods. With this, there is reason to believe that a gamified means of teaching algebra would be a great way to tackle the societal problem presented.

2 Product Overview

Rogue-Like Algebra is a turn-based adventure game that transforms the challenge of learning algebra into an engaging and immersive experience. Designed to address the struggles many face with algebraic concepts, the game blends strategic dungeon-crawling with problem-solving, where players use algebra to attack, defend, and progress. Mistakes carry consequences—such as reduced combat efficiency or character penalties. The game's procedurally generated dungeons and adaptive difficulty ensure replayability and personalized challenge. Aimed primarily at middle and high school students but accessible to all learners, *Rogue-Like Algebra* provides an educational alternative to traditional rote learning by offering a gamified system that rewards creativity, risk-taking, and improvement. The game is available on the web and can be played in any modern browser. The project's goal is to foster confidence in math skills while making algebra fun, approachable, and rewarding.

2.1 Key Product Features and Capabilities

For *Rogue-Like Algebra* to have a smooth development, key objectives and visions for concepts as it pertains to the game must be outlined.

2.1.1 Gameplay

Rogue-Like Algebra will feature staple game mechanics such as a turn-based combat system, an immersive storyline for the player to feel more immersed in the world of the game, and smaller minigames to keep the freshness of the game intact as the player goes through the various processes. It will feature prominent rogue-like mechanics such as permanent death and procedural generation as to make every attempt feel distinct from one another.

2.1.2 Engagement

To keep player retention to a healthy degree, *Rogue-Like Algebra* will feature adaptive means of scaling difficulty to create a sense of challenge for those who look for one, but as to not alienate those who have troubles with learning algebraic concepts. Ways to accomplish this goal include mechanics such as time limits and notable boss battles which can be treated as a checkpoint of sorts.

2.1.3 Objective

The most important objective that *Rogue-Like Algebra* should serve is to gamify and reduce complexity of learning algebraic concepts. The game would accomplish this by different strategies including rewarding players who decide to tackle more difficult problems and giving

players senses of long-term accomplishment by creating various achievements that track throughout the player's entire progress.

3 Major Components

Rogue-Like Algebra will be built by using a variety of tools, systems, and concepts which will be clearly defined below.

3.1 Integrated Development Environment (IDE)

Visual Studio Code – A powerful, extensible, and free coding development environment which will be used as the external code editor for the game engine of choice, Godot.

3.2 Version Control

Git/GitHub – An industry standard of maintaining and documenting the incremental process of development within a team of developers.

3.3 Target Operating System

Windows – Windows is by far the most popular end-user platform, and as such will be kept in mind when developing *Rogue-Like Algebra* to appeal to a wide audience.

Linux – Linux is the operating system that is the backbone of many mobile devices, and as such will be kept in mind when developing *Rogue-Like Algebra* to appeal to a wide audience.

3.4 Game Engine

Godot – Godot is a free and open-source game engine that is very flexible, extensible, and easily accessible with wide-ranging documentation online, leading to it being a very appealing choice when developing *Rogue-Like Algebra*.

3.5 Programming Language

Rust – Rust is a fast, modern, and safe programming language that is quickly rising in popularity, and as such makes an appealing choice for development.

GDScript – GDScript is to be used in Godot to cover and edge cases where Rust is currently unapplicable.

3.6 Systems

Character Controllers – Various character controllers must be created or adapted from freely available templates online.

Leaderboard – A leaderboard system must be created as to provide friendly competitions among users.

Procedural Generators – Procedural generators must be developed for both dungeons and math problems to ensure variety between attempts and as to avoid rote memorization.

Enemy Artificial Intelligence – Various enemy AI must be created to prevent repetitive different kinds of foes for the player to face.

4 Identification of Case Study

For *Rogue-Like Algebra* to succeed in its original mission, the intended demographics must be clearly outlined and kept in mind during development.

4.1 Primary Audience

The aim is to develop *Rogue-Like Algebra* for those who need immediate assistance in the realm of learning algebraic knowledge, particularly those who are enrolled in both middle school and high school, as well as other players in the STEM-oriented field as well as others who enjoy strategic games. Assisting in the learning and reinforcement of foundational algebraic principles in those who struggle to learn said concepts by developing a means that is suggested to be effective methods of teaching would help to accomplish this aim. With this, the hope will be to provide a satisfying and entertaining experience for those who are in the primary audience to retain player retention to satisfy learning progress.

4.2 Secondary Audience

The aim for the secondary audience of *Rogue-Like Algebra* would consist of not only wide audiences of players in various casual genres, but also the parents and educators of those players who are aiming to use *Rogue-Like Algebra* as a means of learning algebraic concepts, as becoming familiar with the product will help to monitor the progress of those learning as well as forming a familiarization for what students enjoy to those who aren't directly playing. For the more casual audience, the goal would be able to reinforce and refresh algebraic concepts and provide a fun and repeatable game loop experience.

5 Glossary

- Adaptive Learning – An educational method that adjusts content difficulty based on the learner’s performance to personalize the learning experience.
- Algebraic Concepts – Foundational mathematical principles involving variables, equations, expressions, and functions typically taught in middle and high school.
- Continuous Integration (CI) – A development practice where developers frequently merge code changes into a shared repository, triggering automated builds and tests to detect issues early.
- Continuous Deployment (CD) – A software release process that automatically delivers validated code changes to production, minimizing manual intervention and ensuring faster updates.
- GDScript – A Python-like scripting language used in the Godot game engine, designed for creating game logic.
- Git – A distributed version control system that tracks changes in source code during software development, enabling collaboration and history tracking.
- GitHub – A cloud-based platform for hosting Git repositories. It offers tools for collaboration, issue tracking, code review, and CI/CD integration.
- Godot – A free and open-source game engine used for developing 2D and 3D games, known for its flexibility and ease of use.
- IDE (Integrated Development Environment) – A software suite that provides tools like a code editor, debugger, and compiler to streamline software development (e.g., VS Code).
- Procedural Generation – A method of creating data algorithmically rather than manually, often used for generating levels, puzzles, or content dynamically in games.
- Rogue-like – A subgenre of games typically featuring procedural generation, turn-based movement, and permanent death, emphasizing strategy and replayability.
- Rust – A systems programming language known for performance, safety, and concurrency.
- Turn-Based Combat – A gameplay mechanic where players and opponents take alternating turns to perform actions such as attacking or defending.
- VS Code (Visual Studio Code) – A popular, lightweight code editor developed by Microsoft, used for writing and debugging code.

6 References

“About Us - DragonBox / WeWantToKnow.” Dragonbox.com, dragonbox.com/about.

Prodigy. “Prodigy Math | Prodigy Education.” [Www.prodigygame.com, www.prodigygame.com/main-en/prodigy-math/](https://www.prodigygame.com/main-en/prodigy-math/).

“IXL - Membership.” [Www.ixl.com, www.ixl.com/membership](https://www.ixl.com/membership).

Steam. “Steam Database.” SteamDB, steamdb.info/.

“Top Roguelike Games by Sales and Revenue on Steam as of 2022 – Steam Marketing Tool.” Games-Stats.com, games-stats.com/steam/?tag=roguelike.

Decker-Woodrow, Lauren E, et al. The Impacts of Three Educational Technologies on Algebraic Understanding in the Context of COVID-19. Vol. 9, 1 Jan. 2023, p. 233285842311659-233285842311659, <https://doi.org/10.1177/23328584231165919>.

Aine, Sandip, and Maxim Likhachev. “Anytime Truncated D* : Anytime Replanning with Truncation.” Proceedings of the International Symposium on Combinatorial Search, vol. 4, no. 1, 20 Aug. 2021, pp. 2–10, <https://doi.org/10.1609/socs.v4i1.18295>

Guy, Stephen, and Ioannis Karamouzas. Guide to Anticipatory Collision Avoidance.

“NavigationMesh.” Godot Engine Documentation, 2025, docs.godotengine.org/en/stable/classes/class_navigationmesh.html

“Creating Your First 2D Game with A* Algorithm.” HackerEarth Blog, 6 Oct. 2016, hackerearth.com/blog/developers/creating-first-2d-game-algorithm/